Camerala: A Browser-Native Framework for Task-Aligned Physiological Feature Logging in Home and Laboratory Settings

Kotaro Furukawa*1

Abstract – **Remote psychophysical experiments increasingly require methods for synchronizing behavioral task events with webcam-derived features while limiting the handling of privacy-sensitive video. We present Camerala, a browser-native, local-first framework that integrates psychophysical task execution, on-device rPPG/FaceMesh-derived feature extraction, frame-level measurement-quality indicators, and timestamped local logging within a unified client-side runtime. The framework avoids raw video transmission and, in the present implementation, stores extracted features and task events locally as CSV files packaged in a ZIP archive. We evaluated Camerala in an exploratory 10-session deployment with one author-participant across one laboratory setting and one home setting. The deployment completed under the tested configuration and produced synchronized task events, sensor-derived features, and measurement-quality indicators. Descriptive results showed differences between the laboratory and home contexts in camera-derived brightness, exposure fluctuation, face-tracking availability, head motion, and rPPG-oriented ROI estimates. Because the study did not include reference physiological measurements, multiple participants, multiple devices, or controlled separation of environmental and task-related factors, these differences should be interpreted as deployment-specific sensor-derived observations rather than evidence of physiological, cognitive, psychological, or motivational effects. The paper contributes a browser-native implementation example and design considerations for task-aligned, local-first physiological feature logging under limited home/laboratory conditions.**

Keywords : **Browser-Native Sensing, Remote Photoplethysmography, FaceMesh, Local-First Logging, Task-Aligned Measurement, Home/Laboratory Deployment**

1. Introduction

Remote or home-based psychophysical experiments often require synchronized behavioral task logs and webcam-derived features while avoiding unnecessary handling of privacy-sensitive video. This use case differs from unconstrained daily-life sensing: the participant is expected to sit in front of a laptop, view task stimuli, and respond through the browser. Even in this task-constrained setting, deployment outside a controlled laboratory introduces practical difficulties for human-interface research, including heterogeneous illumination, camera placement, posture, browser timing, and local data handling.

Many video-based physiological measurement pipelines rely on raw video recording or server-side processing. Such approaches can be difficult to deploy in private spaces because raw facial video is privacy-sensitive and may be identifiable. A local-first browser architecture offers an architectural privacy advantage: raw frames can be processed on the client device, while only extracted features, task events, and quality indicators are stored. This does not constitute a dedicated privacy-preserving algorithm; rather, it is a system-design choice that reduces the need to transmit or retain raw video.

Real-time processing is treated here as a design choice for local-first feature logging rather than as the only technically possible architecture. Recording video locally and processing it offline could preserve timestamps, but it would require storing privacy-sensitive raw video. Camerala instead extracts features and quality indicators during task execution, allowing task events and sensing frames to be logged within the same client-side runtime without raw video persistence. This design also supports online quality checks, such as detecting face-tracking failure, approximate face-size changes, and camera-derived brightness instability during data collection.

To address this deployment scenario, we present **Camerala**, a browser-native framework for task-aligned physiological feature logging in task-

*1: College of Transdisciplinary Sciences for Innovation, Kanazawa University, Japan

constrained home/laboratory experiments.

1.1 Research Questions

We reformulated the exploratory Research Questions (RQs) to focus on operational completion and measurement-quality observations under the tested configuration:

- **RQ1:** Under the tested laptop, browser, camera, and task configuration, can Camerala complete a task-constrained home/laboratory deployment while locally logging synchronized task events, sensor-derived features, and measurement-quality indicators without raw video transmission?
- **RQ2:** What descriptive differences in measurement-quality indicators and sensor-derived estimates are observed between the laboratory and home deployment contexts?

For RQ1, we used the following observation criteria: completion of the planned sessions and trials, local persistence of exported logs, synchronization of task events and frame-level feature logs, retention rate after predefined exclusion criteria, and availability of FPS, face size, face-tracking availability, camera-derived brightness, exposure fluctuation, head motion, and rPPG-oriented ROI estimates. These criteria are intended only to describe operational completion under the tested configuration; they do not demonstrate general deployability across users, devices, or environments.

1.2 Contributions

In this paper, we focus on system integration and conservative deployment observations. The contributions are:

1. **Browser-native local-first sensing architecture**: A client-side pipeline that extracts rPPG/FaceMesh-derived features and measurement-quality indicators without transmitting or storing raw video.
2. **Task-aligned on-device sensing and local persistence**: A unified browser runtime that synchronizes psychophysical task events with frame-level sensor-derived features and stores timestamped local logs.
3. **Exploratory task-constrained home/laboratory deployment**: A limited N=1 author-participant deployment that illustrates operational completion and documents descriptive measurement-quality differences across the tested laboratory and home contexts.

The remainder of this paper is organized as follows. Section II reviews related work in rPPG, web/edge physiological sensing, and task-measurement platforms. Section III describes the Camerala system architecture. Section IV describes the task-constrained deployment and reporting details. Section V presents descriptive observations. Section VI discusses design requirements and safeguards for future deployments, and Section VII concludes.

2. Related Work

2.1 Remote Physiological Measurement and rPPG

Remote photoplethysmography (rPPG) estimates pulse-related color changes from facial video. Classic approaches include ambient-light remote plethysmographic imaging and blind-source video methods [1], [2], while recent methods include deep-learning-based estimators [3]. Prior work has also examined web-deployable, privacy-oriented, and edge-oriented rPPG systems and tools [4]~[6], [8], [9]. These systems show that browser-based and edge-based rPPG functionality already exists. Recent work on online webcam rPPG further shows that uncontrolled capture conditions can affect remote heart-rate imaging [7]. Accordingly, Camerala does not claim to introduce a new rPPG estimator or to solve rPPG degradation under uncontrolled conditions. It records rPPG-oriented ROI values and quality indicators so that task-aligned observations can be inspected and filtered.

2.2 Browser-Based, Edge-Based, and Privacy-Oriented rPPG Systems

Several prior systems are directly relevant to Camerala's positioning. Ayesha *et al.* proposed a web application for experimenting with and validating remote measurement of vital signs [4]. Labvanced provides remote heart-rate detection/rPPG functionality within an online experiment platform [8]. FacePhys-Demo demonstrates browser-based rPPG monitoring with local inference in a public web demo/repository [9]. RhythmEdge addresses contactless heart-rate estimation on edge devices [6]. Gupta and Etemad investigated privacy-preserving remote heart-rate estimation from facial videos [5]. These related systems indicate that browser-based,

web-deployable, edge-based, and privacy-oriented rPPG have already been explored. We therefore avoid claiming novelty in browser-based rPPG functionality, local rPPG processing alone, or privacy preservation as an algorithm. The contribution of Camerala is instead the system-level integration of psychophysical task execution, on-device rPPG/FaceMesh-derived feature extraction, frame-level measurement-quality indicators, and timestamped local persistence within a unified browser-native runtime.

2.3 Task Measurement and Physiological Data Collection Platforms

Well-established web experiment libraries and experiment runtimes, such as jsPsych [10] and PsychoPy/PsychoJS/Pavlovia [11], support browser-based task execution and online experiments. However, they do not generally provide an integrated webcam-derived physiological feature pipeline, task-sensing synchronization, and local-first feature logging as a single browser runtime. Conversely, cloud analytics APIs can provide rich video-based analysis but often require transmitting video streams away from the participant's device. Table 1 maps this design trade-off space.

We structure the initial evaluation as a single-case exploratory deployment. The deployment is not intended to estimate population-level psychological or physiological effects. Instead, it documents whether the integrated browser runtime can complete the planned task-constrained logging procedure and what measurement-quality differences are observed in the tested home/laboratory contexts.

3. System Architecture: Camerala

Camerala (derived from "Camera" + "Laboratory") is a research prototype for task-aligned physiological feature logging in the browser. The contribution is not a new physiological estimator, a method for improving rPPG signal quality, or a dedicated privacy-preserving algorithm. Rather, Camerala is a deployment and logging architecture that combines psychophysical task execution, on-device feature extraction, quality indicators, and local persistence in a single client-side runtime. Although the deployment in this paper used one Gabor-orientation task, the runtime separates task logic from the sensing pipeline so that other task procedures can be implemented without changing the feature-extraction loop.

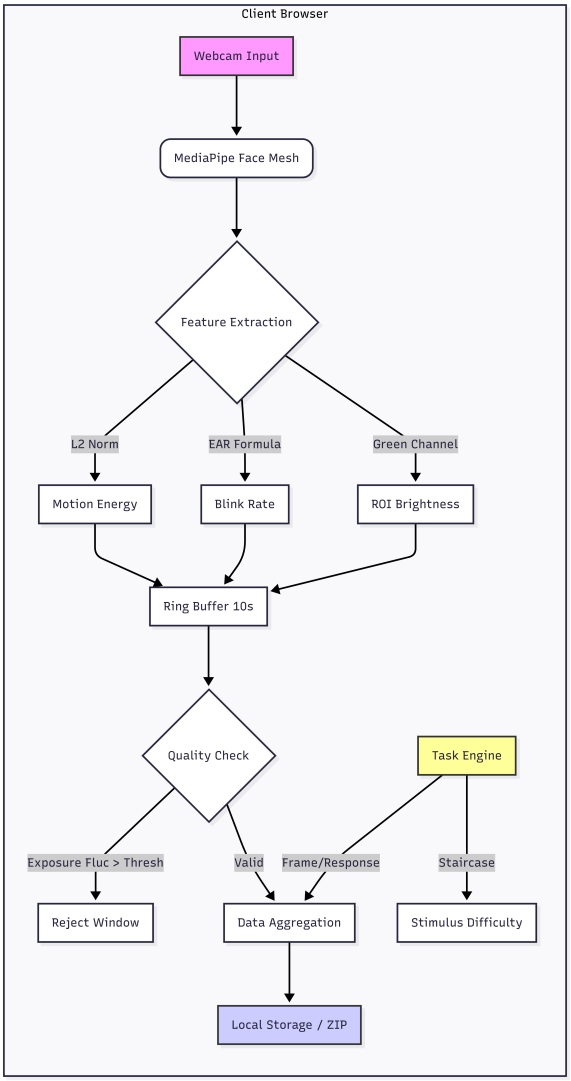

Fig. 1 Overview of the Camerala system architecture. The browser obtains webcam frames, extracts FaceMesh/rPPG-oriented features and quality indicators, synchronizes them with task events, and exports local logs. Under the tested implementation, only extracted features and task events are logged; raw video is neither transmitted nor stored by Camerala.

3.1 High-Level Architecture

The system enables a task-constrained experimental loop inside the browser sandbox (Fig. 1). The architecture consists of three coupled modules:

1. **Task Engine**: Stimulus rendering, trial logic, key responses, feedback, staircase updates, and reaction-time logging.
2. **Vision Pipeline**: Webcam ingestion, MediaPipe Face Mesh inference, rPPG-oriented

Table 1 Design Trade-off Mapping of Data Collection Platforms

System / Platform	On-device video processing	Task-sensing synchronization	Local-first persistence	Raw video export	Typical use
Web experiment libraries	N/A (task only)	Partial	Depends on setup	N/A	Behavioral tasks
Experiment runtimes	Depends on deployment	Yes for task events	Depends on setup	Depends	Lab/online experiments
Browser or web rPPG systems	Often possible	Usually not task-centered	Varies	Varies	Vital-sign estimation
Cloud sensing APIs	No	Low or asynchronous	No	Often required	Video analytics
Camerala	Client-side feature extraction	Task events and frame features in one runtime	Local ZIP export of logs	Not used by Camerala	Task-constrained home/lab studies

Note: The table summarizes design emphasis rather than categorical superiority. Capabilities depend on implementation and deployment configuration.

ROI feature extraction, head-motion estimates, blink/EAR features, and frame-level quality indicators.

3. **Data Manager**: Timestamp alignment, trial/frame aggregation, CSV serialization, ZIP packaging, and local persistence without raw-video storage.

3.2 Technical Implementation: The Vision Pipeline

The current implementation uses MediaPipe Face Mesh, loaded through the browser, to obtain facial landmarks in real time [12]. The camera stream is requested through `getUserMedia` with ideal constraints of 640×480 pixels and 30 fps. Because actual camera settings can depend on the browser and device, these constraints should be interpreted as requested settings rather than verified hardware settings.

3.2.1 Feature Extraction Logic

The present implementation records sensor-derived features and quality indicators rather than validated clinical or medical measurements. Two categories are central to this paper:

1. Head Motion Index (M_t) Head motion is approximated from the frame-to-frame displacement of a facial landmark. Let $L^t = (x^t, y^t)$ be the normalized landmark coordinate at frame t. The motion scalar is computed as:

$$M_t = ||L^t - L^{t-1}||_2. \tag{1}$$

This feature is used as a motion-related quality and behavior indicator. It is not an algorithmic correction for motion artifacts in rPPG.

2. ROI Brightness and Exposure Fluctuation (E_{fluc}) The implementation samples a small forehead region around a FaceMesh landmark and records the green-channel brightness as an rPPG-oriented ROI value. It also computes a frame-level luminance-change indicator:

$$E_{fluc}(t) = |Y_t - Y_{t-1}|, \tag{2}$$

where Y_t is the downsampled frame-luminance average at frame t. Camerala does not attempt to correct rPPG degradation caused by motion or illumination changes. Instead, it records motion-related and brightness-related indicators so that data quality can be inspected and, if necessary, filtered during analysis.

3.3 Technical Implementation: The Task Engine

The experimental psychophysical probe is a two-alternative forced-choice orientation discrimination task. Using the HTML5 Canvas API, the task engine procedurally renders oriented grating stimuli, records key responses, calculates reaction time with `performance.now()`, and updates task difficulty. Procedural rendering avoids network-dependent stimulus loading during a trial and keeps task timing in the same browser runtime as feature logging.

3.3.1 Timing and Synchronization

The implementation uses the following timing strategy:

1. **High-resolution timestamps**: Task events and frame features are timestamped using `performance.now()`.
2. **Frame-loop alignment**: Feature extraction and task rendering are coordinated with the browser frame loop.
3. **Local runtime alignment**: Task events and sensor-derived features are logged in the same browser runtime, avoiding server-side reconcili-

ation.

Algorithm 1 Camerala Task Loop (Simplified)

```
Initialize Camera, FaceMesh, Canvas
while Session Active do
  Block ← GetCurrentBlock()
  Context ← Block.instruction
  ShowInstruction(Context.text)
  for trial i = 1 to n do
    θ ← SampleOrientation({-45°, +45°})
    ShowFixation(400–600 ms)
    t_onset ← performance.now()
    DrawOrientationStimulus(θ, 200 ms)
    while No KeyPress do
      Poll F/J KeyPress
    end while
    t_response ← performance.now()
    RT ← t_response − t_onset
    LogTaskEvent(RT, Accuracy, Context)
    UpdateStaircase(Accuracy, relative ±5% contrast step)
    LogFrameFeatures(FaceMeshFeatures, QualityIndicators)
  end for
end while
```

3.4 Data Management and Local-First Persistence

The current implementation uses JSZip to export local log files as a ZIP archive. The archive contains `metadata.json`, `trials.csv`, `features_raw.csv`, `windows.csv`, `blocks.csv`, and `subjective.csv`. The metadata file stores the session ID, start time, user agent, and block plan. The user initiates the download at the end of the session. This design avoids Camerala-side raw-video storage and avoids raw-video transmission, but it also reduces the ability to inspect image artifacts retrospectively. For this reason, Camerala logs quality indicators such as FPS, approximate face size, face-tracking availability, camera-derived brightness, exposure fluctuation, and head motion.

4. Methodology

4.1 Target Usage Scenario and Exploratory Deployment

The target usage scenario is a remote or home-based psychophysical task performed while seated in front of a laptop, with browser-based logging of behavioral events and webcam-derived features. This is a task-constrained home/laboratory deployment, not continuous physiological sensing during ordinary daily activities such as walking, cooking, conversation, or night-time use.

To evaluate operational completion under this scenario, we conducted an exploratory longitudinal deployment with one author-participant. The deployment consisted of 10 sessions and 600 total trials. Each session consisted of six 10-trial blocks, with two blocks for each instructional task context (neutral instruction, negative instruction, and positive instruction), as reflected in the exported block plans. The order of blocks was randomized by the web application.

4.2 Participant (Autoethnography)

The participant was the primary author. This was a self-experimentation evaluation: no other participants were involved, no identifiable raw video was transmitted by Camerala, and the author provided informed consent for self-participation. Because the participant was also the author and knew that the task instructions were experimental, the instruction blocks cannot be assumed to have induced evaluation pressure, reward motivation, or psychological effects. They are used only as different instructional task contexts for demonstrating synchronized logging across task conditions.

4.3 Hardware and Software Configuration

Table 2 reports the hardware/software configuration used in the deployment.

4.4 Deployment Environment and Lighting Conditions

The deployment was conducted during daytime in an indoor university laboratory/research room and an indoor private home room under general artificial room lighting. Both rooms had windows, but curtains were closed during the sessions. Direct natural light was not used as the primary illumination source in either context. The built-in laptop display brightness was fixed across sessions. Because the deployment used camera-derived brightness values rather than external light-meter measurements, this paper reports brightness as a camera-derived measurement-quality indicator. The camera-derived brightness range was 105–114 in the laboratory setting and 102–111 in the home setting. Table 3 summarizes the deployment environment and lighting conditions.

Table 2 Hardware and software configuration

Item	Reported value
Laptop/PC model	mouse Computer G-Tune P517G60ZO21CNHWT, 15.6-inch gaming notebook
CPU	Intel Core i7-13620H
GPU	NVIDIA GeForce RTX 5060 Laptop GPU
RAM	32 GB
Storage	1 TB SSD
Operating system	Windows 11 Home 64-bit
Browsers used	Mozilla Firefox and Google Chrome
Recorded browser metadata	Available exported metadata include the Firefox user agent already reported in the manuscript. Mozilla Firefox and Google Chrome were used in the deployment.
Webcam type/model	Built-in laptop webcam
Requested camera constraints	`getUserMedia`: ideal 640×480 pixels, ideal 30 fps
Camera capture behavior	Browser-negotiated capture under the requested constraints; no manual exposure or white-balance constraints were applied
Measured processing FPS	Reported in Table 4
Display	Built-in 15.6-inch display, 2560×1440 pixels, 165 Hz
Screen brightness	Fixed across sessions using the built-in laptop display setting
Viewing distance	Approximately 50–60 cm
Auto-exposure	Device/browser default camera pipeline; no manual exposure control was applied
Auto-white-balance	Device/browser default camera pipeline; no manual white-balance control was applied

Table 3 Deployment environment and lighting conditions

Item	Laboratory	Home
Room type	Indoor university laboratory/research room	Indoor private home room
Time of day	Daytime	Daytime
Primary lighting	General indoor artificial lighting	General indoor artificial lighting
Window presence	Windows present	Windows present
Curtain/blind condition	Curtains present and closed	Curtains present and closed
Natural light contribution	Direct natural light was not used as the primary illumination source	Direct natural light was not used as the primary illumination source
Camera-derived brightness range	105–114	102–111
External light-meter measurement	External light-meter values were not used; lighting was characterized using camera-derived brightness and exposure-fluctuation indicators.	
Screen brightness	Fixed across sessions using the built-in laptop display setting.	
Task display	Stimuli were presented on the built-in laptop display. Screen luminance was part of the tested configuration rather than an independently manipulated factor.	
Mean brightness aggregation	Computed from exported camera-derived frame/window logs; aggregated windows used 10-s windows with 5-s overlap.	
Exposure fluctuation aggregation	Frame-level absolute luminance difference, summarized in 10-s windows with 5-s overlap.	
Clouds/screen flicker/power cycles	Not treated as observed experimental events; possible brightness variation was handled through camera-derived brightness and exposure-fluctuation indicators.	

4.5 Experimental Procedure and Instructional Task Contexts

Each session began with a pre-session capture check for basic camera conditions, including approximate camera-derived brightness, pose, and face size, before the camera feed was hidden to reduce distraction. Before each block, the system presented one of three instructional task contexts: neutral instruction, negative instruction, or positive instruction. The negative and positive instruction blocks were not interpreted as validated cognitive or motivational manipulations. They were used to demonstrate that Camerala can synchronize behavioral logs and sensor-derived features across different task con-

texts.

4.6 Psychophysical Task

The task was a two-alternative forced-choice orientation discrimination task using procedurally rendered Gabor stimuli. Each trial began with a randomized fixation period of 400–600 ms, followed by a Gabor stimulus oriented at either $+45°$ or $-45°$ for 200 ms. The participant responded using the F/J keys: the F key corresponded to $-45°$, and the J key corresponded to $+45°$. The application recorded stimulus onset, response key, reaction time, correctness, and task difficulty together with frame-level sensor-derived features. To maintain adaptive task difficulty, the implementation used a standard 1-up 1-down staircase procedure on stimulus contrast. Stimulus contrast was adjusted by relative $\pm 5\%$ steps according to the previous response. These task parameters are reported for reproducibility; the instruction blocks are not interpreted as validated cognitive or motivational manipulations.

4.7 Dependent Measures and Preprocessing

All exported logs were processed sequentially. The primary behavioral measure was reaction time (RT), with an operational analysis range of 100–1500 ms. The primary sensor-derived and quality measures were FPS, face size, face-tracking availability/quality, camera-derived brightness, exposure fluctuation, head motion, and rPPG-oriented ROI brightness. Trials outside the RT range and records below the operational face-tracking threshold were excluded. The face-tracking threshold ($Conf < 0.8$ in the analysis script) was used as a conservative operational exclusion criterion for removing records with low FaceMesh tracking confidence before aggregating frame-derived features. The value 0.8 was selected because FaceMesh confidence is represented on a 0–1 scale and the analysis aimed to retain only records with high face-tracking confidence in this exploratory deployment. This threshold was not independently validated as an rPPG-quality threshold. Therefore, face-tracking availability after filtering should be interpreted only as a FaceMesh/logging quality indicator, not as direct evidence of rPPG signal quality, landmark accuracy, or deployment feasibility.

5. Results

The results are reported descriptively as sensor-derived observations under the tested configuration. The deployment completed 10 sessions, 30 instructional-context groups, and 600 total trials, and the exported local logs were available for analysis. After applying the operational RT and face-tracking exclusion criteria, 582 trials were retained (97.0% retention). This retention rate describes the available analyzed records under the tested configuration and does not demonstrate general feasibility across devices, users, or environments.

5.1 Measurement-Quality Indicators

Table 4 summarizes measurement-quality indicators in the laboratory and home contexts. The camera-derived brightness ranges were similar across the two settings, while the home context showed higher exposure fluctuation in the exported logs. Face-tracking availability remained high after filtering, but this indicator should be read only as a FaceMesh/logging quality indicator, not as validation of rPPG quality or general deployment feasibility.

5.2 Task-Event and Sensor-Derived Observations

To address RQ2, we compared descriptive task-event and sensor-derived summaries across instructional task contexts and deployment contexts. The exported logs showed descriptive differences in head-motion estimates and RT across laboratory and home contexts. These differences should be interpreted as observations from this author-participant deployment, not as evidence that the instruction blocks induced psychological or motivational effects.

In the laboratory context, the negative instruction block showed a lower mean RT than the neutral block in this dataset. In the home context, the mean RT values across instruction blocks were closer together. Because the deployment used one author-participant and did not isolate instruction effects from environment, lighting, posture, or camera factors, these RT patterns are reported only as descriptive task-event observations.

5.3 rPPG-Oriented ROI Estimates

The exported data include rPPG-oriented ROI brightness values and related quality indicators, but no ECG or contact PPG reference was collected.

Table 4 Descriptive measurement-quality comparison

Metric (Mean ± SD unless noted)	Laboratory (292 trials)	Home (290 trials)
Measured processing FPS	59.8 ± 0.4	58.2 ± 2.1
Face size (IOD in px)	142 ± 12	135 ± 18
Camera-derived brightness range	105–114	102–111
Exposure fluctuation (E_{fluc})	0.03 ± 0.01	0.09 ± 0.04
Face-tracking availability after filtering	99.1%	96.5%

Therefore, this paper does not interpret ROI-derived or block-level summaries as validated heart-rate changes. The appropriate interpretation is that the tested deployment yielded sensor-derived estimates and quality indicators whose values differed descriptively between the tested laboratory and home contexts.

6. Discussion

The main implication of this revision is system-design oriented. Camerala illustrates how a browser-native runtime can combine task events, webcam-derived features, quality indicators, and local persistence without raw-video transmission in a task-constrained home/laboratory setting. The results do not establish general deployability, robustness, or physiological/cognitive effects. Instead, they motivate design requirements and safeguards for future local-first deployments.

6.1 Browser-Native Local-First Logging as a Design Pattern

The useful system property demonstrated here is integration. Task timing, frame-level feature extraction, quality indicators, and local export are handled in one client-side workflow. This can reduce the need for raw-video handling and server-side timestamp reconciliation. However, because raw video is not stored, researchers lose the ability to visually inspect many artifacts after data collection. This trade-off makes quality-aware logging especially important.

6.2 Quality Indicators for Task-Constrained Home/Laboratory Deployment

The deployment suggests that future systems should log not only task performance and rPPG-oriented estimates, but also indicators that help interpret the measurement pipeline. Such indicators include FPS, face size, face-tracking availability, camera-derived brightness, exposure fluctuation, and head motion. These values can support pre-session checks, quality flags, and quality-gated analysis. They should not be treated as proof that rPPG estimates are accurate.

6.3 Interpreting Sensor-Derived Estimates Under Environmental Heterogeneity

The laboratory and home contexts showed similar camera-derived brightness ranges, while exposure fluctuation was higher in the home context in the available logs. Because lighting, posture, camera placement, screen luminance, and task context were not independently controlled, the observed differences cannot be attributed to psychological or physiological state. They are better understood as deployment-specific sensor-derived observations under environmental heterogeneity. Future work should include reference ECG or contact PPG, multiple participants, multiple devices, and controlled variation of environmental factors.

6.4 Design Safeguards for Future Camerala Deployments

Future deployments should incorporate clearer safeguards: documented hardware/browser/camera settings, explicit reporting of camera constraints and actual camera settings, fixed screen-brightness settings, pre-session camera/lighting checks, face-size and tracking warnings, camera-derived brightness and exposure-fluctuation flags, and predefined quality-gated analysis rules. If privacy-sensitive raw video is not stored, these safeguards become more important because many artifacts cannot be retrospectively inspected from images. For studies that aim to make claims about physiological responses, a reference physiological sensor and a multi-participant design are necessary.

6.5 Limitations

This study has strict limitations. First, it was an exploratory N=1 deployment with the author as participant. Second, there was no reference physiological sensor, ECG, or contact PPG. Third, the deployment used one primary laptop/camera configuration

and did not test multiple devices, multiple participants, or multiple home environments. Fourth, lighting, posture, camera placement, and task context were not experimentally separated. Fifth, no comparison with existing experiment runtimes, browser rPPG tools, or cloud APIs was conducted. Sixth, the study does not claim clinical or medical accuracy, general deployability, general feasibility, robustness, or validated cognitive, motivational, psychological, or physiological effects. The present study presents an implementation example of browser-native, task-aligned, local-first feature logging under limited home/laboratory conditions.

7. Conclusion

We presented Camerala, a browser-native, local-first framework that integrates psychophysical task execution, on-device rPPG/FaceMesh-derived feature extraction, measurement-quality indicators, and timestamped local persistence. In a limited 10-session N=1 author-participant deployment across one laboratory setting and one home setting, the system completed data collection under the tested configuration without raw video transmission by Camerala. The deployment produced synchronized behavioral logs, sensor-derived features, and quality indicators, and it revealed descriptive differences between the tested environments. These observations should not be interpreted as evidence of general deployability, robustness, or physiological/cognitive effects. Instead, Camerala illustrates a task-aligned browser-native implementation and highlights design safeguards needed for future local-first physiological feature logging in home-based psychophysical experiments.

Data availability

The dataset containing facial landmarks, sensor-derived features, and behavioral responses associated with this study cannot be made publicly available due to privacy restrictions and potentially identifiable face-derived data from a domestic deployment context.

CRediT authorship contribution statement

Kotaro Furukawa: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing - original draft.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence this work.

Funding

This work was supported by the College of Transdisciplinary Sciences for Innovation, Kanazawa University.

Reference

[1] Wim Verkruysse, Lars O. Svaasand, and J. Stuart Nelson. Remote plethysmographic imaging using ambient light. *Optics Express*, 16(26):21434–21445, 2008.

[2] Ming-Zher Poh, Daniel J. McDuff, and Rosalind W. Picard. Non-contact, automated cardiac pulse measurements using video imaging and blind source separation. *Optics Express*, 18(10):10762–10774, 2010.

[3] Weixuan Chen and Daniel McDuff. Deepphys: Video-based physiological measurement using convolutional attention networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 349–365, 2018.

[4] A. H. Ayesha, D. Qiao, and F. Zulkernine. A web application for experimenting and validating remote measurement of vital signs. In *Information Integration and Web Intelligence (iiWAS 2022)*, Lecture Notes in Computer Science, vol. 13635, 2022.

[5] D. Gupta and A. Etemad. Privacy-preserving remote heart rate estimation from facial videos. In *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 706–712, 2023.

[6] Z. Hasan, E. Dey, S. R. Ramamurthy, N. Roy, and A. Misra. Demo: RhythmEdge: Enabling contactless heart rate estimation on the edge. In *2022 IEEE International Conference on Smart Computing (SMARTCOMP)*, pages 177–179, 2022.

[7] D. Di Lernia, G. Finotti, M. Tsakiris, et al. Remote photoplethysmography (rPPG) in the wild: Remote heart rate imaging via online webcams. *Behavior Research Methods*, 56:6904–6914, 2024.

[8] Labvanced. Remote heart rate detection — rPPG. `https://www.labvanced.com/content/technology/en/remote-heart-rate-detection-rPPG/`, accessed July 5, 2026.

[9] K. Wang. FacePhys-Demo: FacePhys web demo. `https://github.com/KegangWangCCNU/FacePhys-Demo`, accessed July 5, 2026.

[10] J. R. de Leeuw. jsPsych: A JavaScript library for creating behavioral experiments in a Web browser. *Behavior Research Methods*, 47(1):1–12, 2015.

[11] J. W. Peirce, J. R. Gray, S. Simpson, M. MacAskill, R. Hoechenberger, H. Sogo, E. Kastman, and J. K. Lindelov. PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, 51:195–203, 2019.

[12] C. Lugaresi, J. Tang, H. Nash, et al. MediaPipe: A framework for building perception pipelines. arXiv:1906.08172, 2019.

Appendix

Supporting Example: Task Substitution

A core engineering feature of the framework is the separation between task logic and the feature-extraction loop. Listing 1 illustrates a simplified configuration for replacing the orientation task with a spatial-memory task while preserving the same logging pathway. This example is intended to illustrate module separation rather than to provide additional validation data.

Listing 1: Task substitution example

```
const task = {
  stimulus: renderSpatialGrid(memory_load_target),
  allowed_keys: ["Space"],
  on_frame_sync: (timestamp_ms) => {
    camerala_logger.logEvent("stimulus_onset", timestamp_ms);
  }
};
```

Listing 2 shows a schema-style example of a local tabular log in which task events and frame-derived features can coexist through timestamps.

Listing 2: Example local log schema

```
time_ms,event,load,rppg_roi,head_motion
4015.2,grid_on,4,110.4,0.02
4016.8,frame_sync,4,109.8,0.02
4032.5,frame_sync,4,111.1,0.01
4211.4,response,4,112.0,0.01
```